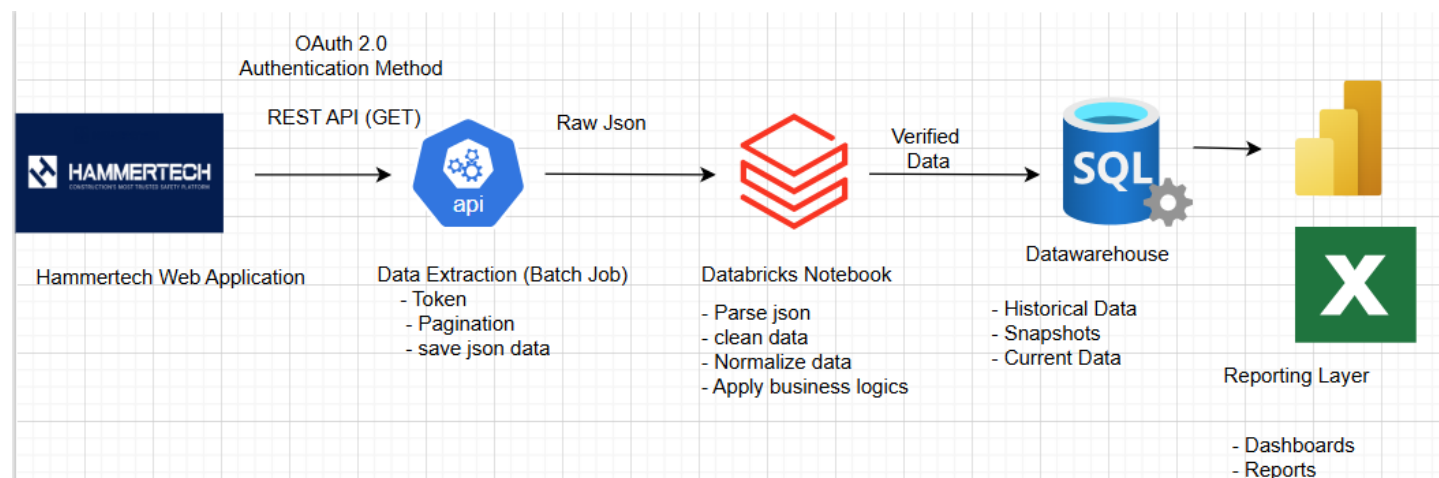


Extracting data from Hammertech

Stakeholders: Business Analysts, Safety Department, Risk Management Department, leadership team members

Purpose: The main purpose of this project is to address some of the key problems regarding the compliance, regulations, training as per Occupational Safety and Health Administration (OSHA) and safety challenges during the construction project. It is very critical for safety department and risk management departments of the construction firms to be aware of those key factors regarding the OSHA guidelines and maintain compliance with the standards and regulations to avoid any forms of penalties

High-level Architecture Diagram of the Technical Workflow



Technical Summary:

Cadence: This data architecture pipeline will run every hour during during the Weekdays.

Hammertech Web Application: Hammertech is a construction-based environment, health and safety (EHS) software solution. This software application offers the construction management companies to supervise and manage the site access, safety, and security control of the construction projects. Hammertech helps the project stakeholders in reducing the operational costs, improve safety and reduce the insurance premiums, avoid lawsuits and expenses via managing all the important areas of operations and safety in a single platform.

API: To extract data using API, Hammertech's official documentation has been used in the entire project. To obtain the bearer token using the Basic authentication method the official documentation [Link](#) has been used.

Authentication:

The first step is to generate an API Key which will be used to authenticate all the REST API requests.

Key Generation:

To generate the key, we need to login to the Hammertech System and go to the My Account option and select the API Keys from the drop down menu. We need to create a new API Key and then save the generated key which will be used in the data extraction method.

Authentication Method:

Pseudo code:

Basic Access:

```
Authorization: 'Basic ' + base64.b64encode(f'{{<clientid>}}:{{<client secret key>}}'.encode()) .decode()
```

Pagination:

Many Reporting API endpoints can return large result sets. To help you work efficiently and avoid timeouts or memory issues, the API uses pagination. This means results are returned in chunks ("pages"), and you can request the next page using a link provided in the response.

Different endpoints use different pagination approaches. The sections below explain how they work and what to expect when integrating with them.

Server-Driven Paging

Some endpoints automatically apply paging on the server. These endpoints have a default page size which is also the maximum page size. Refer table below for exact details. If more data is available, the response includes an @odata.nextLink property. You can call the URL in nextLink to retrieve the next page.

Hammertech Reporting API:

Attendees

Request:

```
GET: https://example.hammertechonline.com/v0.1/attendees
```

```
import requests

url = "https://example.hammertechonline.com/v0.1/attendees"

headers = {"accept": "application/json"}

response = requests.get(url, headers=headers)

print(response.text)
```

Response

 200
OK

```
{
  "@odata.context": "string",
  "value": [
    {
      "AttendeeId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "InspectionId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "AttendeeName": "string",
      "AttendeeCompany": "string",
      "AttendeeEmail": "string",
      "AttendeeUserId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "InspectionTypeName": "string",
      "InspectionTypeId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "InspectionDateTime": "2026-04-25T18:43:18.433Z",
      "InspectionCreatorUserName": "string",
      "InspectionCreatorUserId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "ProjectId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "ProjectName": "string",
      "ProjectState": "string",
      "LastModifiedDate": "2026-04-25T18:43:18.433Z",
      "InspectionCompletedDate": "2026-04-25T18:43:18.433Z"
    }
  ]
}
```

Incidents

Request:

```
GET: https://example.hammertechonline.com/v0.1/incidents
```

```
import requests

url = "https://example.hammertechonline.com/v0.1/incidents"

headers = {"accept": "application/json"}

response = requests.get(url, headers=headers)

print(response.text)
```

Response

 200

OK

```
{
  "@odata.context": "string",
  "value": [
    {
      "IncidentId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "ProjectId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "Project": "string",
      "ProjectName": "string",
      "IncidentTypeId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "IncidentTypeName": "string",
      "IsNearMiss": "string",
      "IsIncidentInvolveDeath": "string",
      "IsIncidentInvolveMedicalTreatmentByDoctor": "string",
      "IsIncidentInvolveImmediateHospitalTreatment": "string",
      "IsIncidentInvolveHighRiskEquipmentOrPlant": "string",
      "IsIncidentInvolveTheftOfExplosives": "string",
      "IsIncidentInvolveInjuriesCausedByExplosives": "string",
      "IsIncidentInvolveReleaseOfDangerousGoods": "string",
      "IsIncidentInvolveNoneofabove": "string",
      "NeedToBeReportedToRelevantAuthority": "string",
      "ReferenceNumber": "string",
      "Name": "string",
    }
  ]
}
```

```
"PositionTitle": "string",
"PhoneNumber": "string",
"DateAdded": "2026-04-25T18:43:18.433Z",
"DateEdited": "2026-04-25T18:43:18.433Z",
"DateOfIncident": "2026-04-25T18:43:18.433Z",
"DescriptionOfIncident": "string",
"EmployerInChargeOfEquipmentId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
"EmployerInChargeOfEquipment": "string",
"EmployerProfileIdInChargeOfEquipmentId": "3fa85f64-5717-4562-b3fc-
2c963f66afa6",
"EmployerProfileIdInChargeOfEquipment": "string",
"EmployerNameInChargeOfEquipment": "string",
"EmployerInjuredPersonString": "string",
"EmployerInjuredPerson": "string",
"EmployerInjuredPersonId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
"EmploymentType": "string",
"WorkActivites": "string",
"InjuryDescription": "string",
"Witness": "string",
>ActionTaken": "string",
"IncidentLocation": "string",
"IsReportable": "string",
>DateReported": "2026-04-25T18:43:18.433Z",
"SummaryOfEvents": "string",
"PreliminaryInvestigation": "string",
"EmployerNameInjuredPerson": "string",
"EmployerProfileIdInjuredPerson": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
"EmployerProfileInjuredPerson": "string",
"CustomFields": "string",
>UserSpecifiedReferenceNumber": "string",
"Reference": "string",
>Status": "string",
"IsConfidential": "string",
>LastModifiedDate": "2026-04-25T18:43:18.433Z",
"Employers": "string",
"Workers": "string",
"IsDeleted": "string",
"ClosedByUsername": "string",
"ClosedByUserId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
>DateClosed": "2026-04-25T18:43:18.433Z",
>DateAdvisedOfEvent": "2026-04-25T18:43:18.433Z"
}
]
}
```

Injuries

Request:

GET: <https://example.hammertechonline.com/v0.1/injuries>

```
import requests

url = "https://example.hammertechonline.com/v0.1/injuries"

headers = {"accept": "application/json"}

response = requests.get(url, headers=headers)

print(response.text)
```

Response

 200

OK

```
{
  "@odata.context": "string",
  "value": [
    {
      "InjuryId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "ProjectId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "Project": "string",
      "ProjectName": "string",
      "EmployerId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "EmployerProfileId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "EmployerName": "string",
      "StartingTime": "string",
      "StartTime": "string",
      "EndTime": "string",
      "ShiftArrangement": "string",
      "MoreThan8Hours": "string",
      "JobTitle": "string",
      "MainTasks": "string",
      "TrainingProvided": "string",
      "DateAdded": "2026-04-25T18:43:18.433Z",
      "DateEdited": "2026-04-25T18:43:18.433Z",
      "DateTimeOfInjury": "2026-04-25T18:43:18.433Z",
      "TreatmentType": "string",
    }
  ]
}
```

```

    "FollowupFirstAidTreatmentRequired": "string",
    "DateAttendedMedicalCentre": "2026-04-25T18:43:18.433Z",
    "IsMedicalCertificateReceived": "string",
    "DateMedicalCertificateReceived": "2026-04-25T18:43:18.433Z",
    "CompletedByName": "string",
    "CompletedByPositionTitle": "string",
    "IncidentId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "Incident": "string",
    "IsLostTimeInjury": "string",
    "ShiftsLost": 0,
    "InjuryBodilyLocations": "string",
    "InjuryNatures": "string",
    "InjuryMechanisms": "string",
    "InjuryAgencies": "string",
    "InjuryBreakdownAgencies": "string",
    "CustomFields": "string",
    "ReferenceNumber": 0,
    "Reference": "string",
    "WorkerId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "IsConfidential": "string",
    "LastModifiedDate": "2026-04-25T18:43:18.433Z",
    "InjuryOtherTreatmentType": "string",
    "InjuredPersonType": "string",
    "WorkerActionsAtTheTime": "string",
    "WhatHappenedUnexpectedly": "string",
    "HowWasInjurySustained": "string",
    "IsDeleted": "string",
    "AdditionalDetails": "string",
    "InjuredPersonName": "string",
    "DescriptionOfOccuranceOfInjuryOrDisease": "string",
    "CommentsOnTreatmentProvidedByFirstAidOfficer": "string"
  }
]
}

```

Databricks Notebook

Databricks Notebook is used to read the json data that is obtained from the web api using the GET method. In the databricks notebook various python and pyspark codes are used to convert the json into tabular data, clean the data, apply business logic and then validate the data. Once the data is validated then it is loaded into the Azure Sql database.

Sample code using DUMMY DATA SETS

```

import os
import json
import requests
from datetime import datetime, timezone

# -----
# CONFIG (Key Vault Secrets)
# -----
HAMMERTECH_BASE_URL = os.getenv(
    "HAMMERTECH_BASE_URL",
    "https://example.hammertechonline.com"
)
API_TOKEN = os.getenv("HAMMERTECH_API_TOKEN", "API_TOKEN")

# Incremental watermark storage
WATERMARK_FILE = "/tmp/hammertech_injuries_watermark.txt"

# -----
# WATERMARK HANDLING
# -----
def read_watermark():
    if not os.path.exists(WATERMARK_FILE):
        return "2000-01-01T00:00:00Z"
    with open(WATERMARK_FILE, "r") as f:
        return f.read().strip()

def write_watermark(value_iso):
    with open(WATERMARK_FILE, "w") as f:
        f.write(value_iso)

# -----
# API CALL (OData compliant)
# -----
def fetch_injuries_incremental(modified_since_iso):
    headers = {
        "Accept": "application/json",
        "Authorization": f"Bearer {API_TOKEN}"
    }

    # Example OData parameters
    params = {
        "$filter": f"ModifiedDate gt {modified_since_iso}",
        "$orderby": "ModifiedDate asc"
    }

```



```

        # $top, $skip could also be used if needed
    }

    url = f"{HAMMERTech_BASE_URL}/v0.1/injuries"

    response = requests.get(url, headers=headers, params=params, timeout=60)
    response.raise_for_status()

    return response.json()

def main():
    last_watermark = read_watermark()
    injuries = fetch_injuries_incremental(last_watermark)

    # Update watermark after successful extraction
    new_watermark = datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")
    write_watermark(new_watermark)

    raw_output_path = "/tmp/hammertech_injuries_raw.json"
    with open(raw_output_path, "w") as f:
        json.dump(injuries, f)

    print(f"Extracted {len(injuries)} injury records")
    print(f"Raw data written to {raw_output_path}")
    print(f"Watermark updated to {new_watermark}")

if __name__ == "__main__":
    main()

```

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, when, upper, to_date,
    current_timestamp, row_number
)
from pyspark.sql.window import Window

spark = SparkSession.builder.appName("HammerTech_Injuries_ETL").getOrCreate()

# -----
# READ RAW JSON
# -----
raw_path = "/tmp/hammertech_injuries_raw.json"

```

```

df_raw = spark.read.json(raw_path)

# -----
# DATA CLEANING
# -----
df_clean = (
    df_raw
    .withColumn("injury_type", upper(col("InjuryType")))
    .withColumn("incident_date", to_date(col("IncidentDate")))
    .withColumn("modified_date", to_date(col("ModifiedDate")))
    .withColumn("worker_name", upper(col("WorkerName")))
    .dropDuplicates()
)

# -----
# BUSINESS LOGIC
# -----
df_logic = (
    df_clean
    .withColumn(
        "severity_category",
        when(col("Severity") >= 4, "CRITICAL")
        .when(col("Severity") >= 2, "MODERATE")
        .otherwise("MINOR")
    )
    .withColumn(
        "recordable_flag",
        when(col("Recordable") == True, "YES")
        .otherwise("NO")
    )
    .withColumn("etl_loaded_at", current_timestamp())
)

# -----
# DEDUPLICATION
# -----
window_spec = (
    Window
    .partitionBy("InjuryId")
    .orderBy(col("ModifiedDate").desc())
)

df_final = (
    df_logic
    .withColumn("rn", row_number().over(window_spec))
)

```

```

        .filter(col("rn") == 1)
        .drop("rn")
    )

    # -----
    # WRITE TO AZURE SQL DATABASE
    # -----
    jdbcHostname = "yourserver.database.windows.net"
    jdbcPort = 1433
    jdbcDatabase = "yourdatabase"
    jdbcUsername = "sqladmin"
    jdbcPassword = "YourPassword123!"

    jdbcUrl = (
        f"jdbc:sqlserver://{jdbcHostname}:{jdbcPort};"
        f"database={jdbcDatabase};encrypt=true;"
        "trustServerCertificate=false;"
        "hostNameInCertificate=*.database.windows.net;"
        "loginTimeout=30;"
    )

    connectionProperties = {
        "user": jdbcUsername,
        "password": jdbcPassword,
        "driver": "com.microsoft.sqlserver.jdbc.SQLServerDriver"
    }

    target_table = "dbo.hammertech_injuries_curated"

    (
        df_final.write
        .mode("append")
        .jdbc(url=jdbcUrl, table=target_table, properties=connectionProperties)
    )

    print(f"Loaded {df_final.count()} curated injury records into {target_table}")

```

Datawarehouse: All the verified data is now loaded in the Azure Sql database. Azure Sql database works as a Datawarehouse where all the historical, current and snapshot data is stored for all the reporting purposes.

Reporting Layer: Reporting tools such as Power BI and Microsoft Excel are used to create reports and dashboards using the data stored in the tables of the Azure Sql database.

References:

- <https://reportingdocs.hammertech.com/reference/authentication-1>
- https://reportingdocs.hammertech.com/reference/mhammertechsafetymanagerreportingapicontrollersv0_1inspectionattendeessv01controllergetsystemwebodataqueryodataqueryoptionshammertechsafetymanagerreportingapimodelsv0_1inspectionattendeexportlistitem
- https://reportingdocs.hammertech.com/reference/mhammertechsafetymanagerreportingapicontrollersv0_1incidentsv01controllergetsystemwebodataqueryodataqueryoptionshammertechsafetymanagerreportingapimodelsv0_1incidentexportlistitemssystembooleansystemnullablesystemguidssystemnullablesystemdatetime
- https://reportingdocs.hammertech.com/reference/mhammertechsafetymanagerreportingapicontrollersv0_1injuriesv01controllergetsystemwebodataqueryodataqueryoptionshammertechsafetymanagerreportingapimodelsv0_1injuryexportlistitemssystembooleansystemnullablesystemguidssystemnullablesystemdatetime